Project Learning Guide v3

Every file in `hdl_sources/`: what it does and how it works

Real-Time OV7670 Video Capture and VGA Processing System

What changed from v2. Version 2 focused on the big concepts with readable images. Version 3 keeps that style, but adds a file-by-file explanation of all HDL and constraint files in `hdl_sources/`.

Important note about the generated images. The diagrams in this PDF are included as generated PNG image files, not drawn as tiny LaTeX charts. The editable SVG versions are also saved in `report/images/`.

Figure 1: All files in `hdl_sources/` and the best order to read them.

Quick System Review

Before reading individual files, keep the whole system in mind:

- Camera configuration path: `ov7670_config.v -> sccb_master.v -> OV7670 registers`.
- Camera video path: `OV7670 -> ov7670_capture.v -> frame_buffer.v`.
- VGA display path: `frame_buffer.v -> display_scaler.v -> filter_engine.v -> VGA pins`.
- Timing path: `vga_controller.v` generates screen counters and sync pulses.
- Physical board path: `constraints.xdc` maps Verilog ports to FPGA pins.

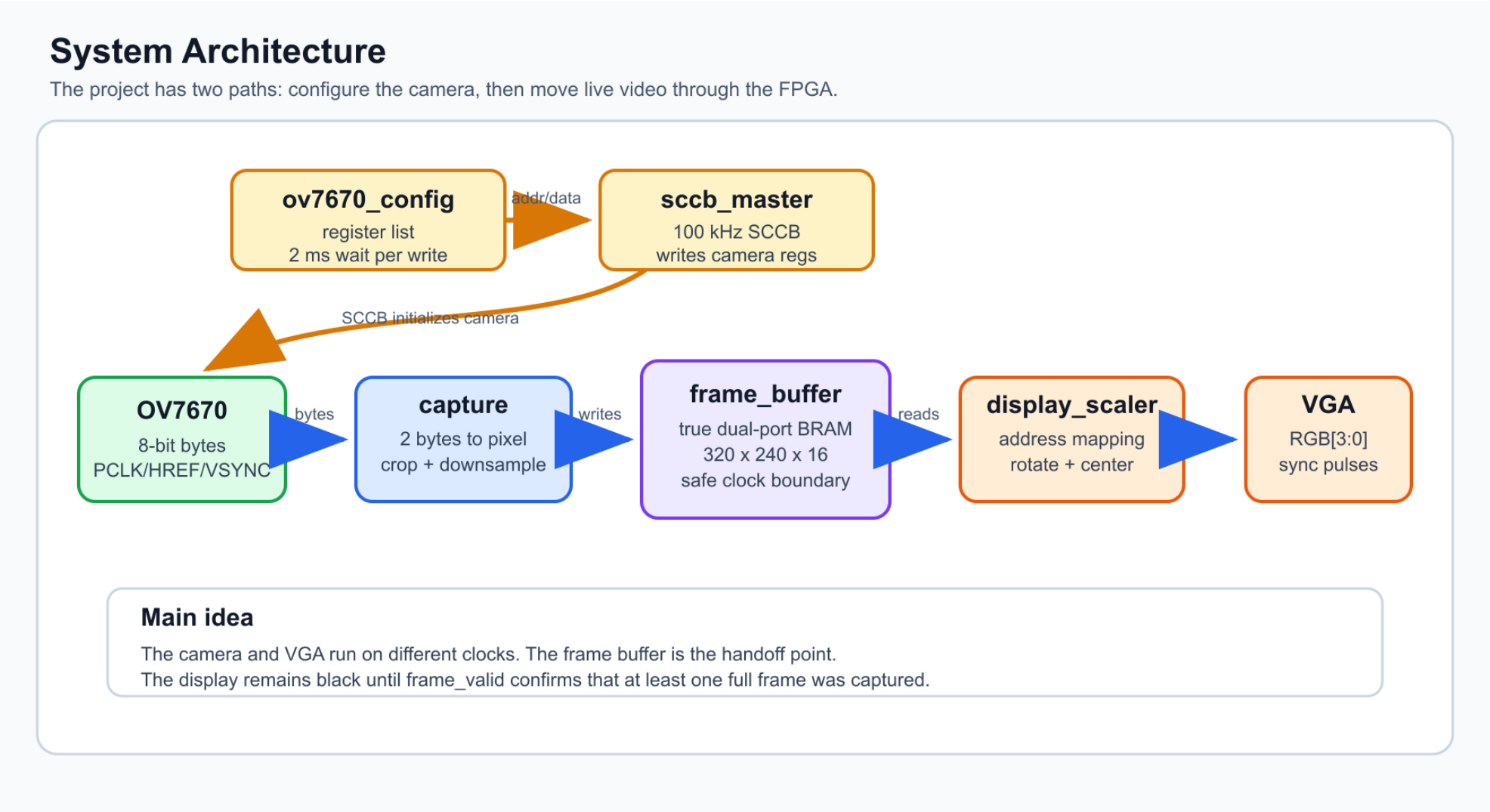

Figure 2: Readable full-system architecture carried forward from v2.

top.v

What it does — System integration module

`top.v` is the highest-level Verilog file. It does not perform the detailed pixel algorithm itself. Its job is to instantiate and connect every major block: clock generation, camera configuration, camera capture, frame buffer, VGA timing, display scaler, filter engine, reset logic, and clock-domain crossing logic.

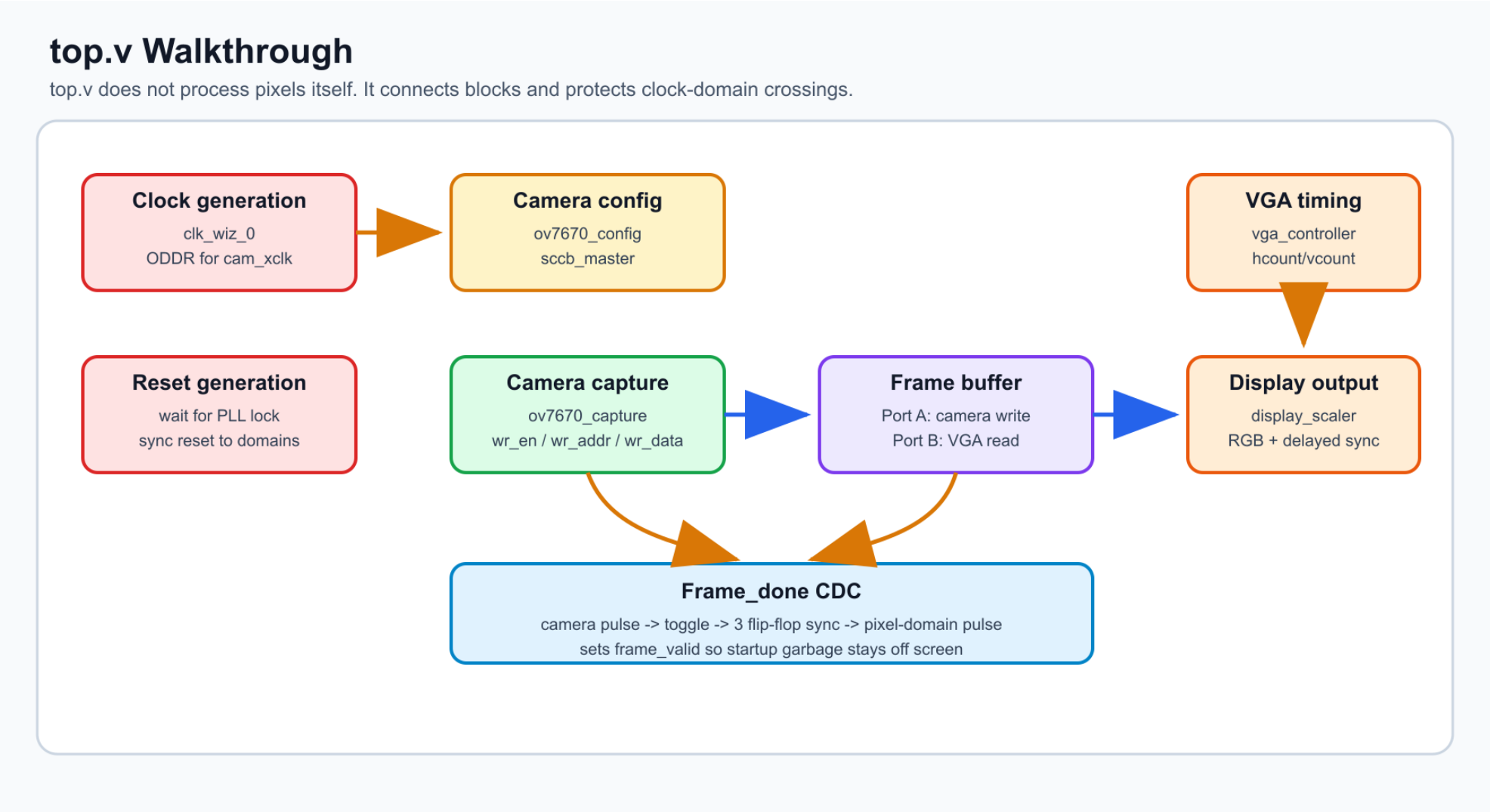

Figure 3: How `top.v` connects the rest of the design.

How it works

- Instantiates `clk_wiz_0`; output 1 becomes `pixel_clk`, and output 2 is sent to the camera clock output path.
- Uses an ODDR primitive to drive `cam_xclk` out of the FPGA pin with a stable duty cycle.

- Drives `cam_pwdn = 0` and `cam_rst_n = 1`, so the OV7670 is enabled and not held in hardware reset.
- Creates `global_rst` with a 24-bit counter after the PLL locks, then synchronizes reset into the pixel and camera domains.
- Connects `ov7670_config` to `sccb_master`; `sccb_master` drives `cam_scl` and `cam_sda`.
- Connects `ov7670_capture` writes to BRAM Port A, and display reads to BRAM Port B.
- Converts the one-cycle camera-domain `frame_done` pulse into a toggle, synchronizes it into `pixel_clk`, and sets `frame_valid`.
- Delays VGA sync outputs so the monitor sync aligns with delayed BRAM/display pixel data.

Key CDC idea:

```
always @(posedge cam_pclk)
    if (frame_done) frame_done_toggle <= ~frame_done_toggle;

always @(posedge pixel_clk)
    fd_sync <= {fd_sync[1:0], frame_done_toggle};

assign frame_done_pix = fd_sync[2] ^ fd_sync[1];
```

`constraints.xdc`

What it does — Board pin and clock constraint file

`constraints.xdc` is not Verilog. It tells Vivado which physical FPGA pins connect to each top-level port in `top.v`. It also declares clock timing constraints and asynchronous clock groups.

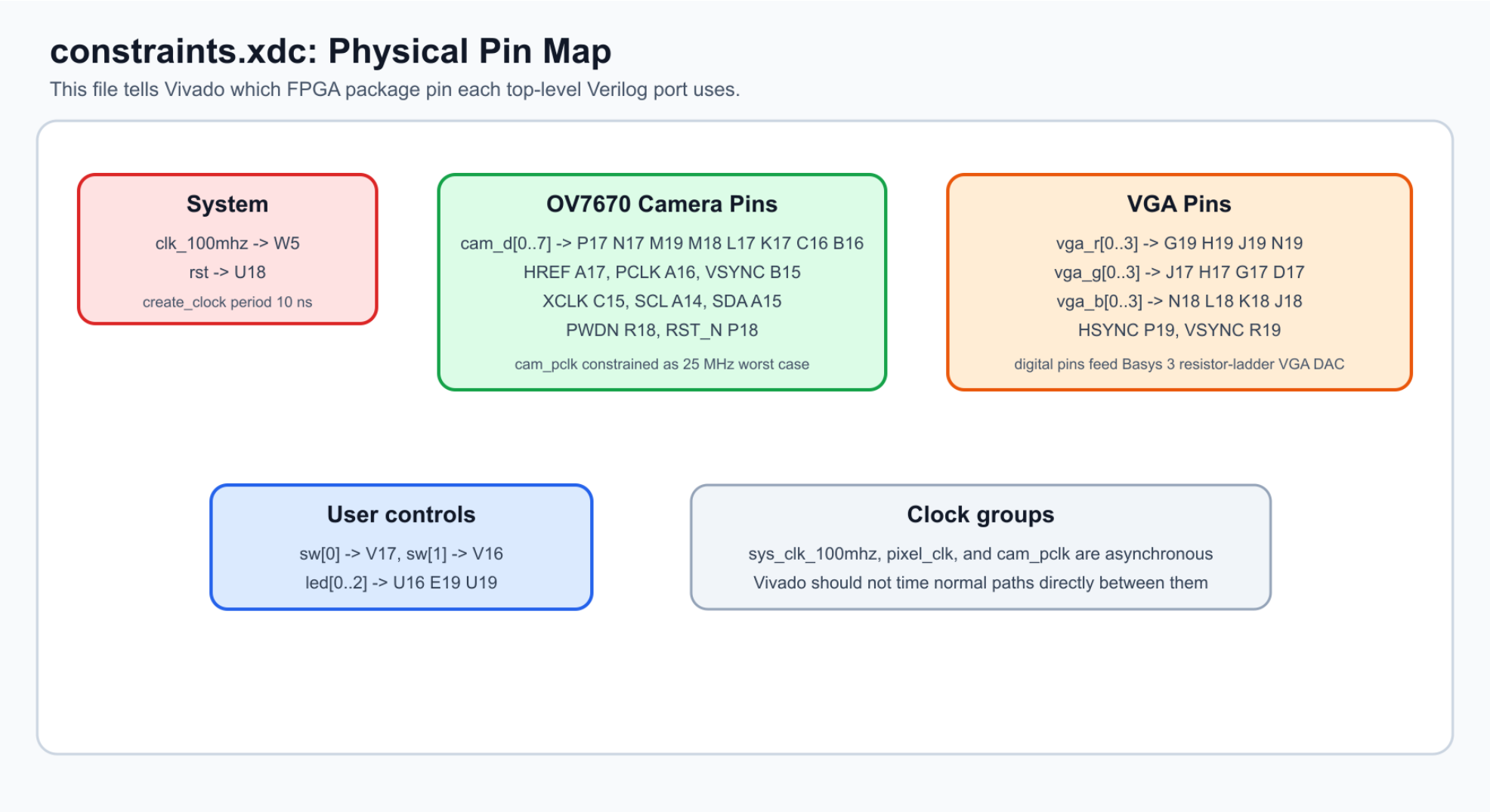

Figure 4: Physical pin map and timing constraints from `constraints.xdc`.

How it works

- Maps the 100 MHz board clock to pin W5 and creates a 10 ns clock constraint.
- Maps reset to BTNC on U18.

- Maps OV7670 data, timing, SCCB, reset, power, and clock pins.
- Maps VGA RGB and sync pins.
- Maps `sw[1:0]` to slide switches used by the filter engine.
- Declares `cam_pclk` as a separate camera clock.
- Marks system, pixel, and camera clocks as asynchronous groups, so Vivado does not assume normal timing relationships between them.

ov7670_config.v

What it does — OV7670 register-write sequencer

`ov7670_config.v` stores the camera setup table and feeds one register address/data pair at a time to `sccb_master.v`. The camera needs these writes before the RGB565 video stream is useful.

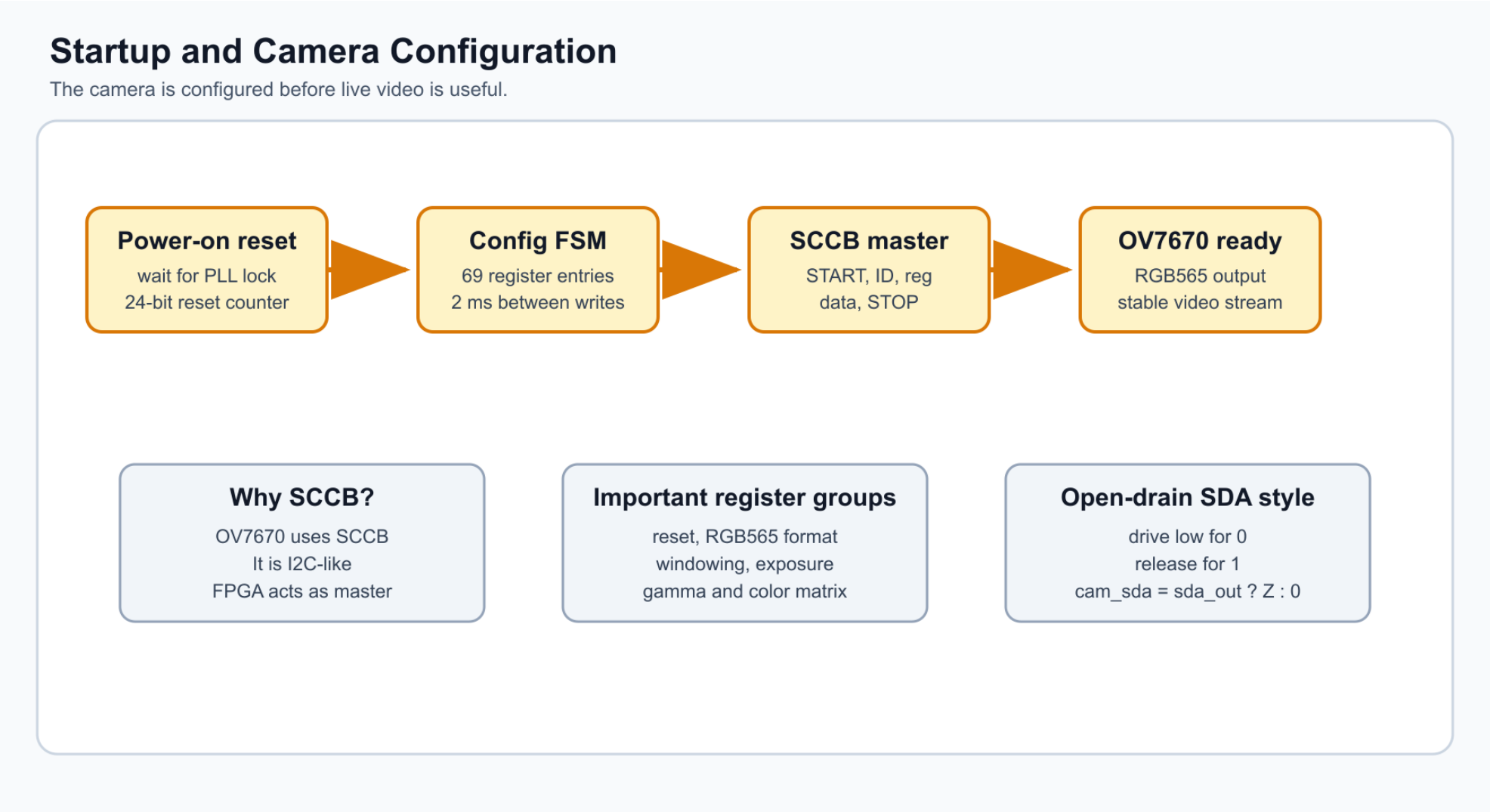

Figure 5: Camera startup and SCCB configuration path.

How it works

- Contains a ROM-style `case (rom_idx)` with 69 register/data pairs.
- Important register groups include soft reset, clocking, RGB565 format, image windowing, color matrix, exposure/gain/white balance, gamma,

and stability options.

- Uses a four-state FSM: wait, start transaction, wait for SCCB done, done.
- Waits `RESET_WAIT = 200_000` cycles between writes. At 100 MHz, that is 2 ms.
- Pulses `sccb_start` for one clock when a register write should begin.
- Raises `cfg_done` after the last register entry.

Core timing:

```
localparam RESET_WAIT = 200_000; // 2 ms at 100 MHz

S_START_TX: begin
    reg_addr   <= current_reg[15:8];
    reg_data   <= current_reg[7:0];
    sccb_start <= 1'b1;
    state      <= S_WAIT_DONE;
end
```

sccb_master.v

What it does — SCCB serial write engine

`sccb_master.v` converts one requested camera register write into actual SCCB pin activity. SCCB is the OV7670 control bus and is similar to I2C.

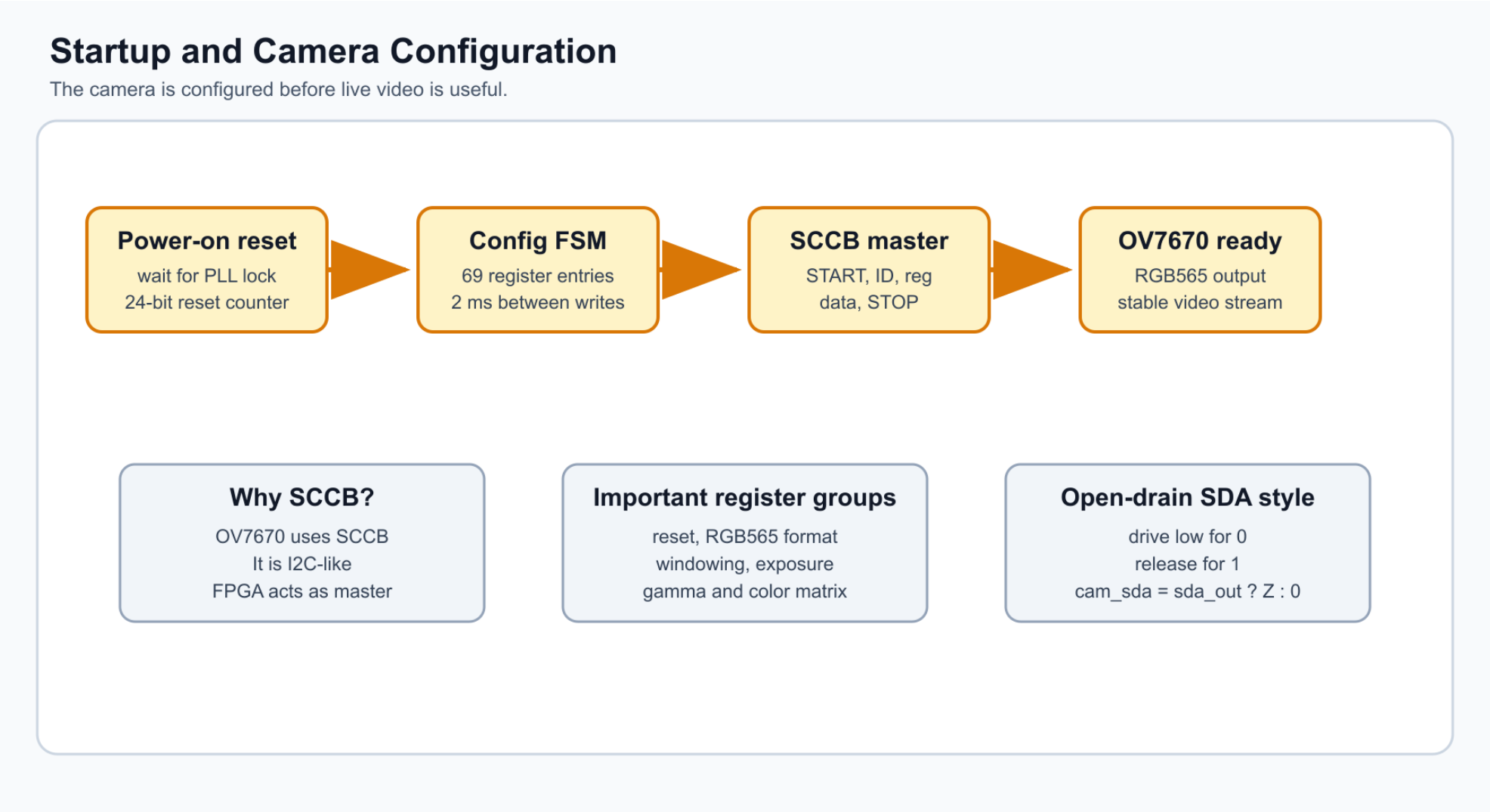

Figure 6: The SCCB master is the serial bus block in the camera configuration path.

How it works

- Divides the 100 MHz system clock down to a 100 kHz SCCB clock.
- Uses a quarter-period phase counter so SDA changes while SCL is low and is stable while SCL is high.

- Sends START, ID byte, don't-care/ACK phase, register address, don't-care phase, data byte, don't-care phase, and STOP.
- The OV7670 7-bit device address is `7'h21`; the transmitted write byte is `8'h42`.
- Outputs `done` for one clock when the transaction finishes.
- `top.v` turns its SDA output into open-drain style behavior: drive low for 0, otherwise release to high impedance.

Transaction shape:

```
START -> {dev_addr, 1'b0} -> reg_addr -> reg_data -> STOP
```

Clock divider:

```
localparam CLK_DIV = CLK_FREQ / (SCCB_FREQ * 4);
```

ov7670_capture.v

What it does — Camera byte capture and BRAM write generator

`ov7670_capture.v` receives the OV7670 parallel video stream. It combines two 8-bit transfers into one RGB565 pixel, removes unstable edge pixels, downsamples the frame, and produces BRAM write signals.

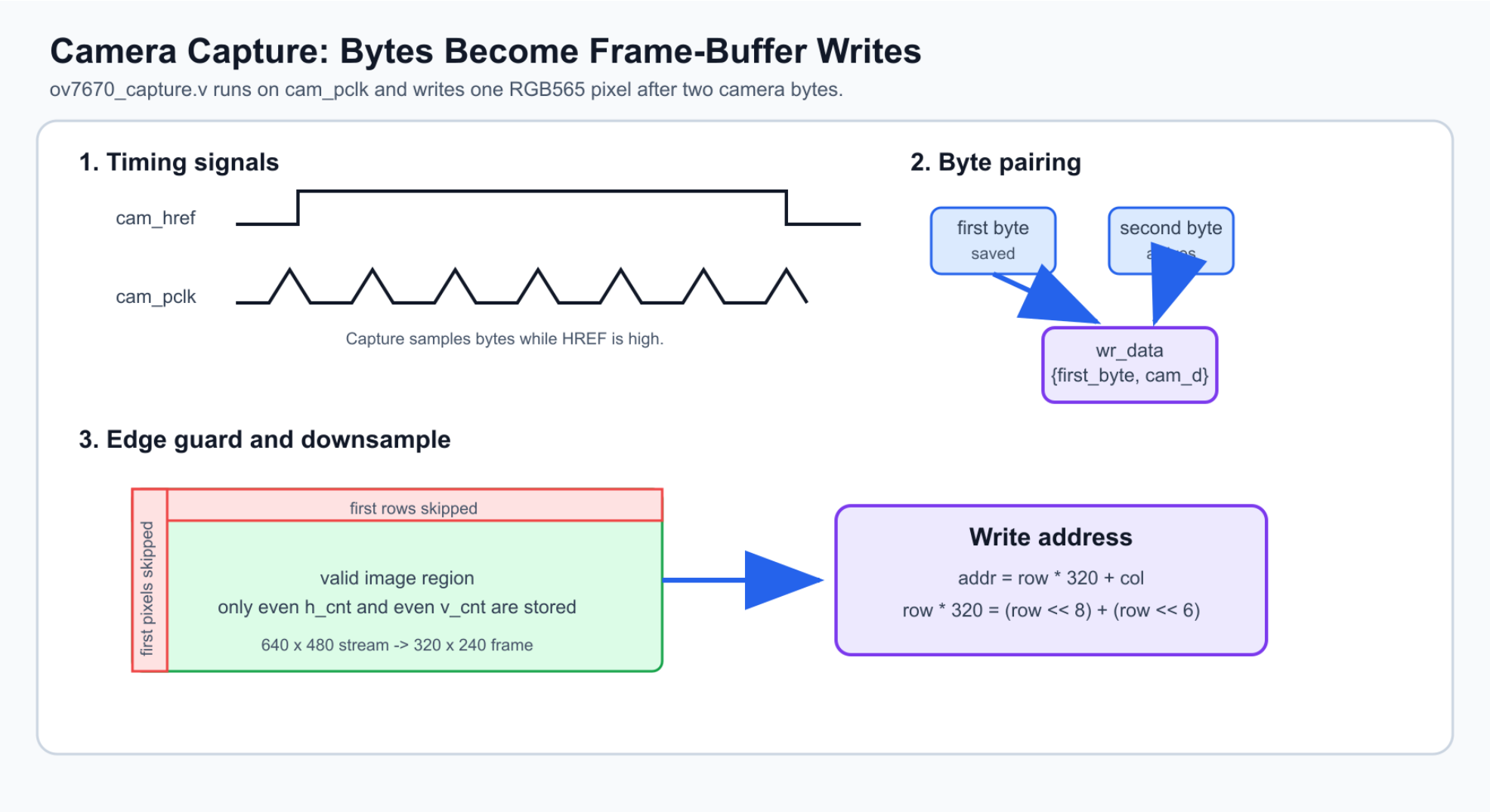

Figure 7: How camera bytes become RGB565 BRAM writes.

How it works

- Runs on `negedge pclk`, the camera pixel byte clock.
- Uses `byte_cnt` to remember whether the current byte is the first or second byte of a pixel.

- Stores the first byte in `first_byte`; on the second byte, creates `wr_data = {first_byte, cam_d}`.
- Uses `cam_href` to know when a row is valid.
- Uses `cam_vsync` to reset frame counters and pulses `frame_done` at the end of VSYNC.
- Skips the first 4 rows and first 4 pixel pairs to avoid edge noise.
- Keeps only even horizontal and vertical positions, reducing 640 x 480 into 320 x 240.
- Computes BRAM address as row times 320 plus column, implemented with shifts.

Important code:

```
if (h_cnt[0] == 1'b0 && v_cnt[0] == 1'b0) begin
    wr_data <= {first_byte, cam_d};
    wr_en   <= 1'b1;
    wr_addr <= (row << 8) + (row << 6) + col;
end
```

`frame_buffer.v`

What it does — Dual-port frame memory wrapper

`frame_buffer.v` wraps the Xilinx Block Memory Generator IP. It gives the camera side a write port and the VGA side a read port.

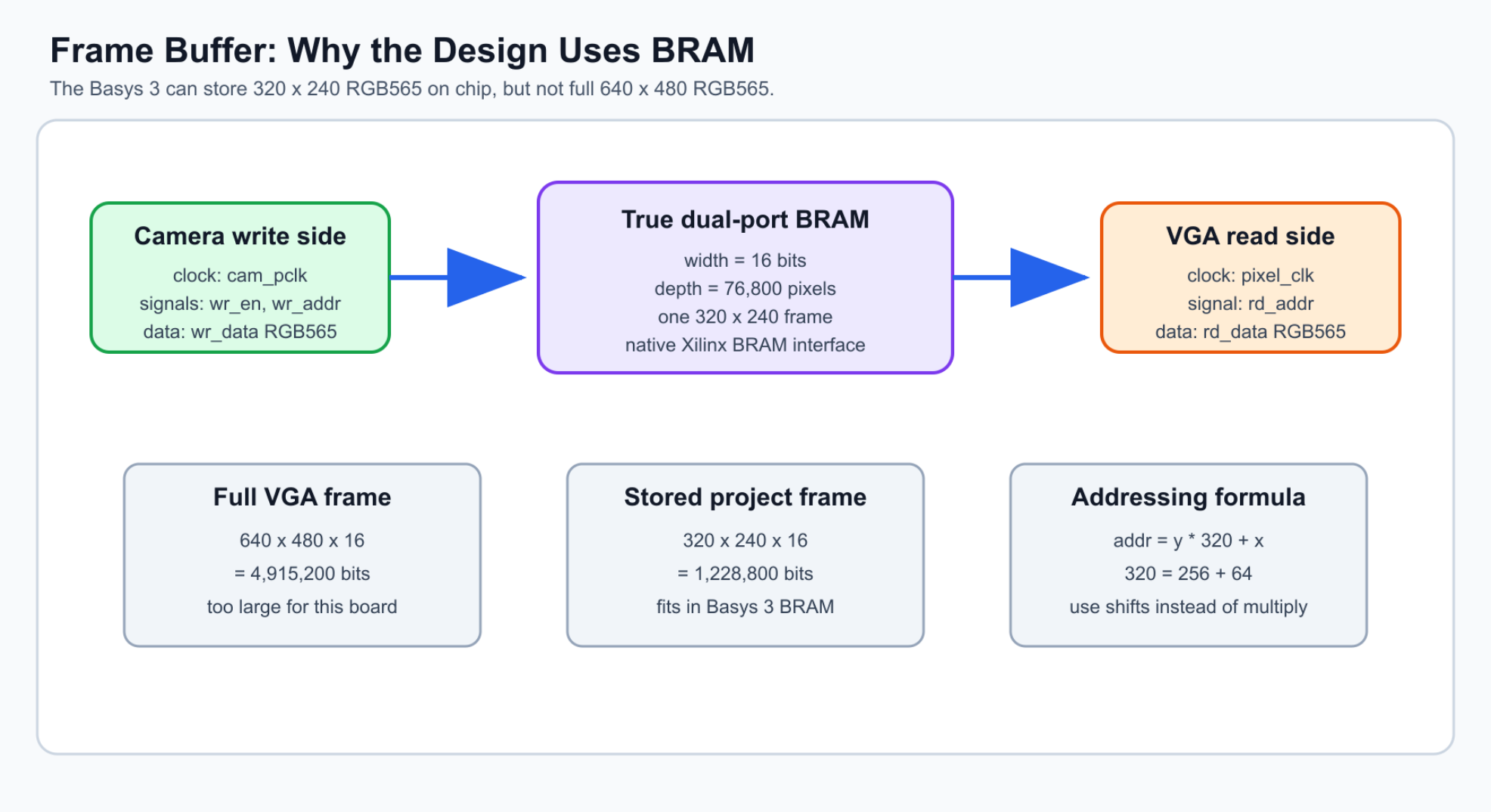

Figure 8: Why the project uses a 320 x 240 RGB565 dual-port BRAM frame buffer.

How it works

- Port A uses `clka = cam_pclk`; it writes when `wea` is high.
- Port B uses `clkb = pixel_clk`; it continuously reads from `addrb`.
- The memory is 76,800 entries deep and 16 bits wide.
- Under `SIMULATION`, the file uses a behavioral Verilog array so Cocotb can test without the Xilinx IP.
- In synthesis, it instantiates `blk_mem_gen_0`.

Main reason:

```
320 * 240 * 16 = 1,228,800 bits
```

That fits in Basys 3 BRAM, while a full 640 x 480 RGB565 frame does not.

vga_controller.v

What it does — 640 x 480 VGA timing generator

vga_controller.v creates the screen counters and sync pulses required by a VGA monitor.

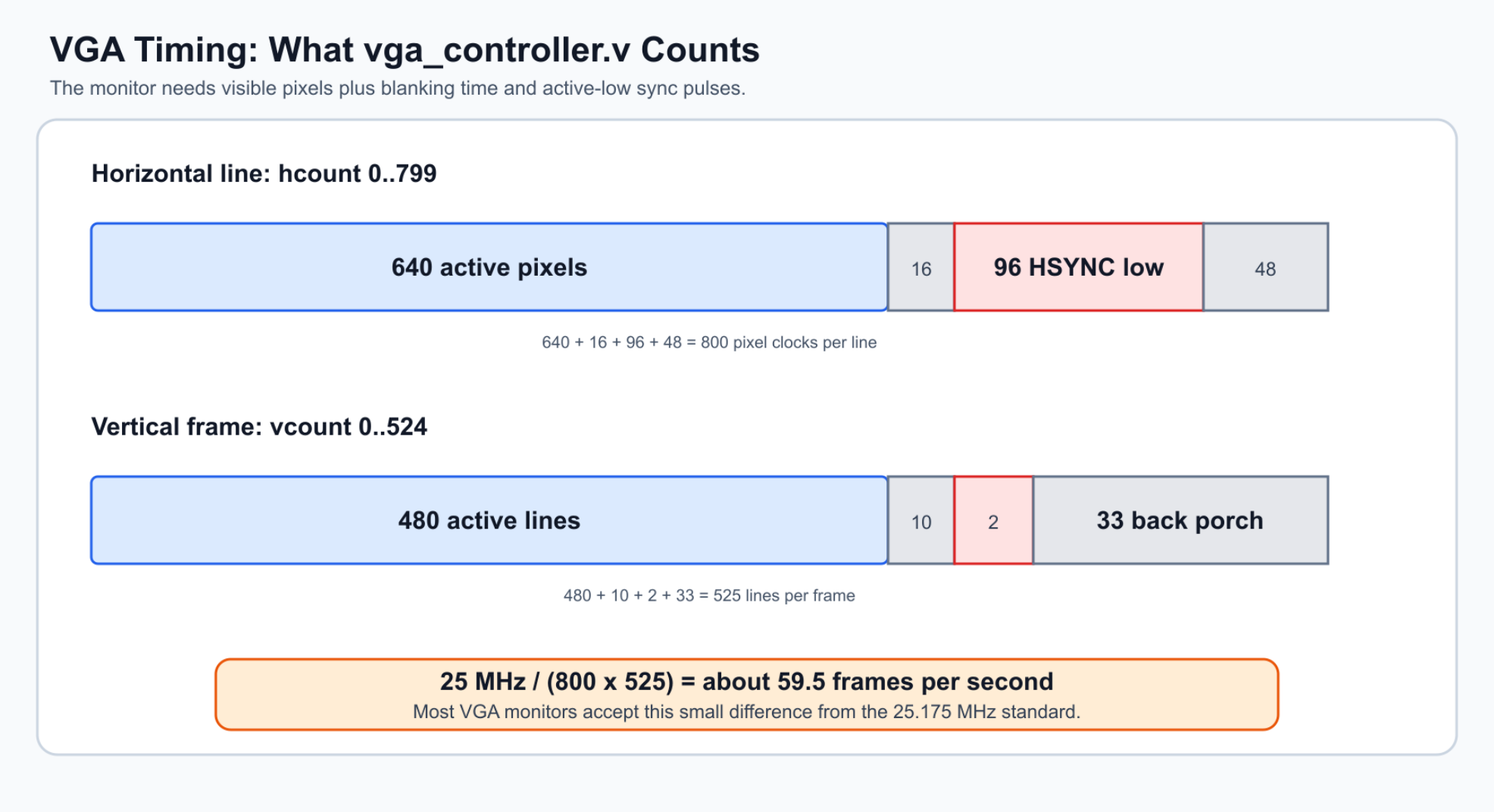

Figure 9: VGA timing counters and visible/sync regions.

How it works

- `hcount` increments every pixel clock from 0 to 799.
- When `hcount` wraps, `vcount` increments from 0 to 524.
- `active` is high only when `hcount < 640` and `vcount < 480`.
- HSYNC is active low from horizontal count 656 through 751.
- VSYNC is active low for vertical rows 490 and 491.
- The monitor sees 640 x 480 visible pixels plus porch and sync periods.

Key logic:

```
assign hsync  = ~((hcount >= H_SYNC_START) && (hcount < H_SYNC_END));
assign vsync  = ~((vcount >= V_SYNC_START) && (vcount < V_SYNC_END));
assign active = (hcount < H_ACTIVE) && (vcount < V_ACTIVE);
```

display_scaler.v

What it does — VGA screen coordinate to BRAM address mapper

`display_scaler.v` decides which stored camera pixel should be shown at the current VGA pixel position. It also gates output until the first complete frame exists.

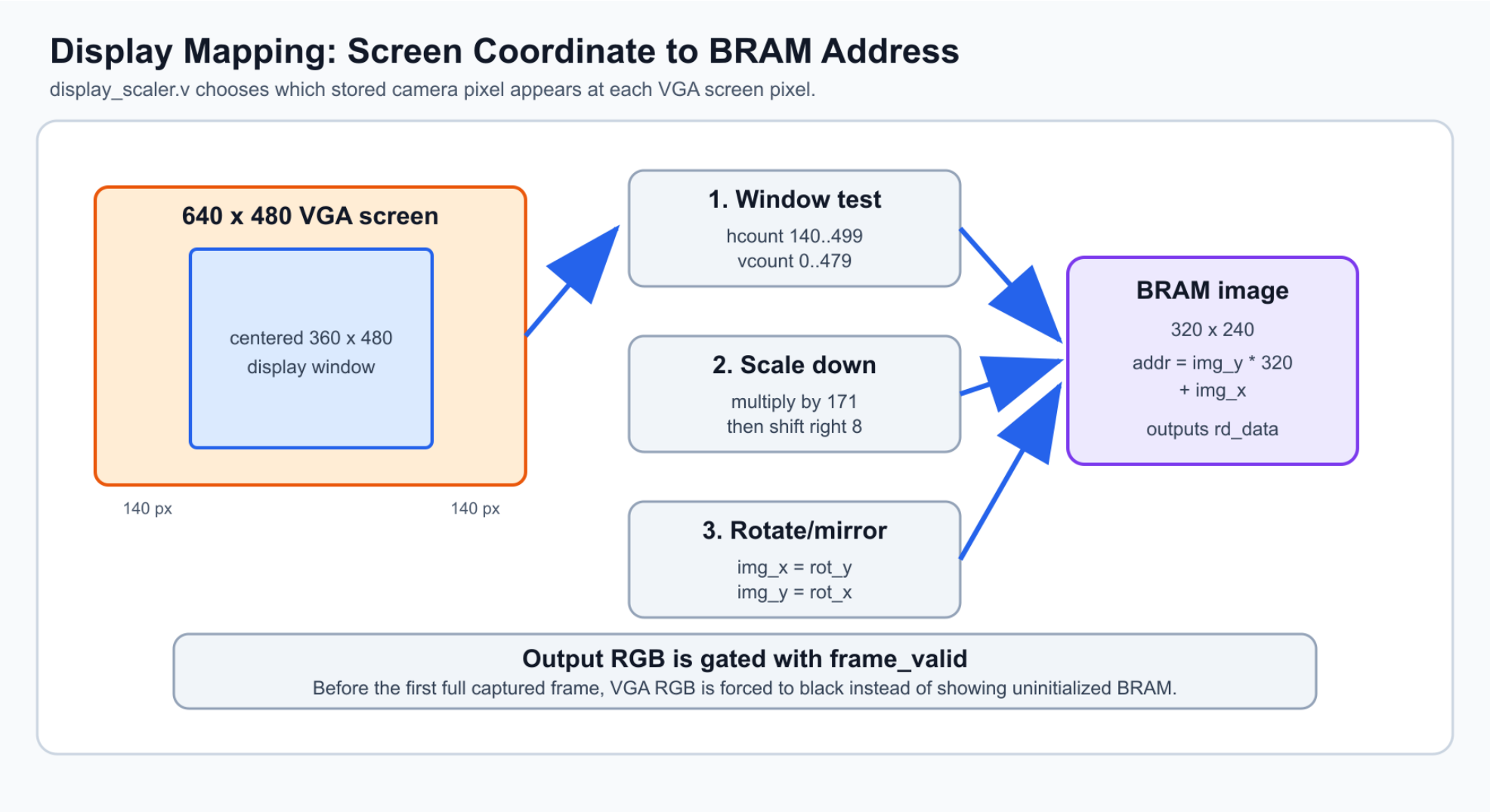

Figure 10: How `display_scaler.v` maps screen coordinates to frame-buffer addresses.

How it works

- Receives `hcount`, `vcount`, and `active` from `vga_controller.v`.
- Displays only in the centered screen region where `hcount` is 140 through 499 and `vcount < 480`.

- Converts the 360 x 480 screen region down to a 240 x 320 rotated coordinate using a 171/256 fixed-point scale.
- Sets `img_x = rot_y` and `img_y = rot_x`, which handles the rotate/mirror correction for this camera mounting.
- Computes `rd_addr = img_y * 320 + img_x`.
- Delays the active flag two cycles to align with BRAM read latency.
- Sends `rd_data` through `filter_engine`.
- Drives VGA RGB from the high bits of filtered RGB565.

Important mapping:

```
screen_x = hcount - 10'd140;
rot_x    = (screen_x * 8'd171) >> 8;
rot_y    = (vcount   * 8'd171) >> 8;
img_x    = rot_y;
img_y    = rot_x;
```

`filter_engine.v`

What it does — Combinational RGB565 pixel filter

`filter_engine.v` changes one RGB565 pixel at a time based on `sw[1:0]`. It has no clock and no frame memory.

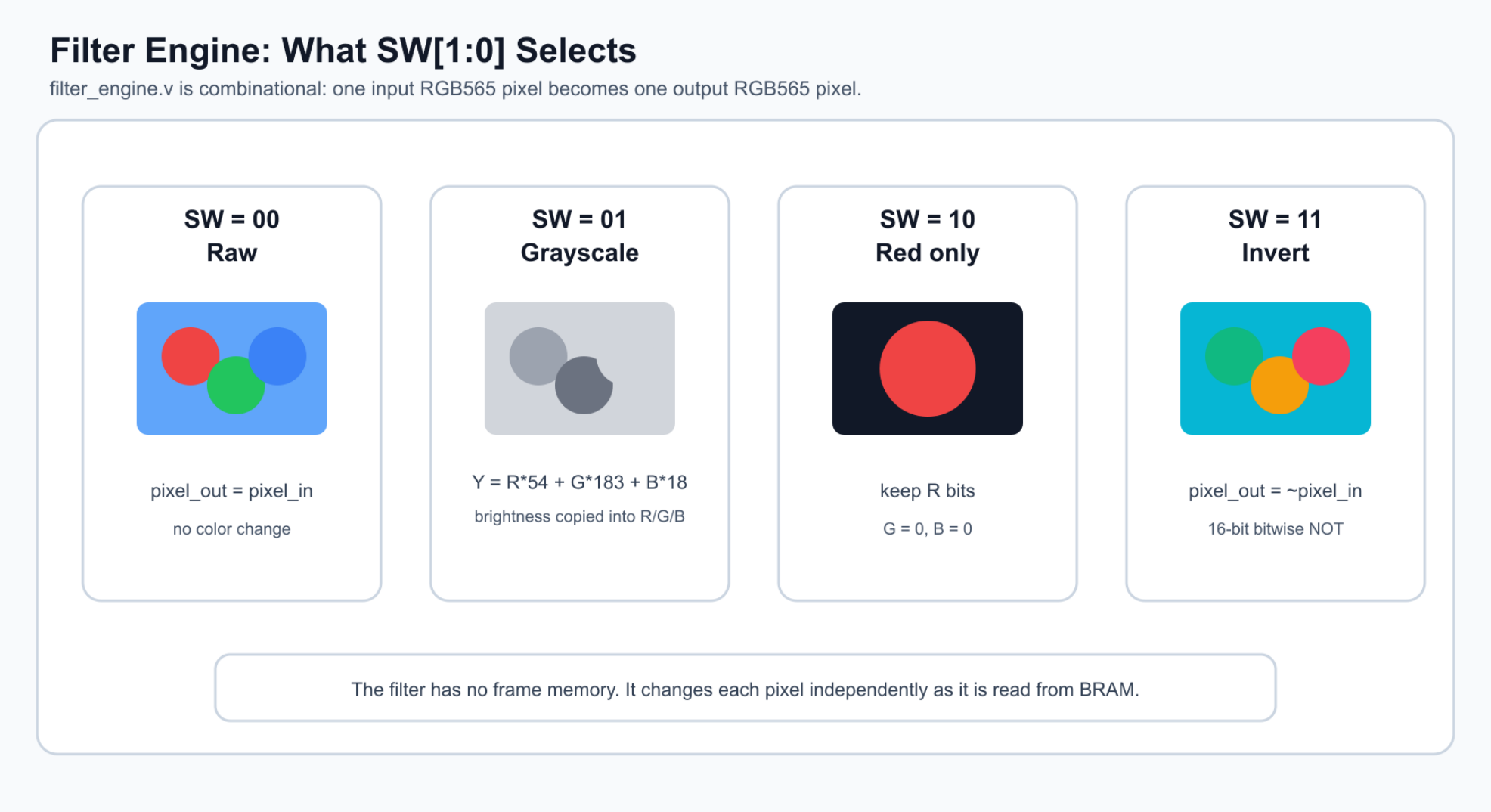

Figure 11: Switch-selected filter modes.

How it works

- Splits the input pixel into `r5 = pixel_in[15:11]`, `g6 = pixel_in[10:5]`, and `b5 = pixel_in[4:0]`.
- `sw = 00`: raw pass-through.
- `sw = 01`: grayscale using integer weighted sum.
- `sw = 10`: red-only by keeping red and clearing green/blue.
- `sw = 11`: inversion by bitwise NOT of the 16-bit input.

Grayscale logic:

```
y_scaled = (r5 * 54) + (g6 * 183) + (b5 * 18);
y_6bit   = y_scaled[13:8];
y_5bit   = y_6bit[5:1];
pixel_out = {y_5bit, y_6bit, y_5bit};
```

How the HDL Files Work Together

Figure 12: Use this order when reading or debugging the HDL files.

If this fails	Check these files first
Camera does not output useful pixels	`top.v`, `constraints.xdc`, `ov7670_config.v`, `sccb_master.v`.
Pixels are not written to memory	`ov7670_capture.v`, then `frame_buffer.v`.
VGA sync works but image is black	`display_scaler.v`, `frame_buffer.v`, and `frame_valid` logic in `top.v`.
Filter switches do not work	`filter_engine.v`, switch pins in `constraints.xdc`, and `sw` wiring in `top.v`.

Final Summary

The project works because each HDL file has a narrow responsibility:

- `constraints.xdc` connects the abstract Verilog ports to real Basys 3 pins.
- `top.v` connects all hardware modules and protects clock-domain crossings.
- `ov7670_config.v` and `sccb_master.v` make the camera output the desired RGB565 stream.
- `ov7670_capture.v` converts camera bytes into downsampled frame-buffer writes.
- `frame_buffer.v` stores a full 320 x 240 RGB565 frame and bridges camera/VGA clocks.
- `vga_controller.v` creates the monitor timing.
- `display_scaler.v` maps screen coordinates back to stored camera pixels.
- `filter_engine.v` changes pixel color based on the switches.

Main idea to remember: The frame buffer is the center of the system. The camera writes it at camera timing, and VGA reads it at monitor timing.